

An Introduction to Reinforcement Learning

Caleb Lee, Catherine Cao, Arshia Mago, Ashlee King, Christina Fan^a

^a*University of Waterloo, Department of Psychology, 200 University Avenue
West, Waterloo, N2L 3G1*

Abstract

This is the abstract. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum augue turpis, dictum non malesuada a, volutpat eget velit. Nam placerat turpis purus, eu tristique ex tincidunt et. Mauris sed augue eget turpis ultrices tincidunt. Sed et mi in leo porta egestas. Aliquam non laoreet velit. Nunc quis ex vitae eros aliquet auctor nec ac libero. Duis laoreet sapien eu mi luctus, in bibendum leo molestie. Sed hendrerit diam diam, ac dapibus nisl volutpat vitae. Aliquam bibendum varius libero, eu efficitur justo rutrum at. Sed at tempus elit.

Keywords: Q-learning, Reinforcement learning, State, Agent, Action, TD learning

1. Introduction

1.1. History

- Ashlee History

1.2. Importance and Applications

- Arshia + some Catherine applications stuff <!-- Here are two sample references: [Feynman and Vernon Jr. \(1963\)](#) [Dirac \(1953\)](#).

With this template using elsevier class, natbib will be used. Three bibliographic style files (*.bst) are provided and their use controlled by `cite-style` option:

- `citestyle: number` (default) will use `elsarticle-num.bst` - can be used for the numbered scheme
- `citestyle: numbername` will use `elsarticle-num-names.bst` - can be used for numbered with new options of natbib.sty
- `citestyle: authoryear` will use `elsarticle-harv.bst` — can be used for author year scheme

*Corresponding author

This `citestyle` will insert the right `.bst` and set the correct `classoption` for `elsarticle` document class.

Using `natbiboptions` variable in YAML header, you can set more options for `natbib` itself. Example

```
yaml natbiboptions: longnamesfirst,angle,semicolon ->
```

2. Mathematical Theory

2.1. Markov Decision Process

The Markov Decision Process (MDP) forms the mathematical foundation of reinforcement learning (RL) algorithms. It provides a framework for modeling sequential decision-making problems in stochastic environments, where an agent's actions influence both immediate rewards and subsequent states. These systems satisfy Markov property, meaning that the state transition and reward depend only on the current state and action, regardless of the prior process.

A MDP is defined by states, actions and rewards, which together describe the interaction between an agent and its environment. The set of all possible states is denoted by $\mathcal{S}$, where each state S_t represents specific coordinates or condition of the environment at time step t . The agent observes the current state S_t and selects an action A_t , where $\mathcal{A}(s)$ denotes the set of available actions in that state.

After taking action A_t , the agent transitions to a new state S_{t+1} . The state transition probability is the likelihood of transitioning to a subsequent state s' given the current state-action pair (s, a) :

$$P(S' | s, a) = \Pr(S_{t+1} = S' | S_t = s, A_t = a)$$

Simultaneously, the agent receives an expected immediate reward $R(s, a)$, which is defined as the expected reward received after taking action A in state S :

$$R(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$$

2.2. Policy and Value Function

A policy, denoted as π , specifies agent's behaviour by defining how actions are selected in each state, $\pi(a | s) = \Pr(A_t = a | S_t = s)$ represents the probability that the agent selects action a when in state s . The state-value function, $v_\pi(s)$, represents the expected cumulative reward received when the agent starts in state s and follows policy π . It is defined as:

$$v_\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

where $\gamma \in [0, 1]$ is the discount factor, which balances immediate reward and the long-term reward. A smaller γ gives more weight to immediate rewards, while a closer to 1 gives more weight to long-term rewards.

To evaluate the value of specific actions taken in a state, we use the action-value function (or Q-function), $q_\pi(s, a)$. This function measures the expected cumulative reward when the agent starts in state s , takes action a , and follows policy π :

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

2.3. Bellman Equation

The Bellman equation for v_π shows a recursive relationship between the value of a state and the values of subsequent states. It describes the value of a state as the sum of the expected immediate reward and the discounted value of subsequent states. For any policy π , the state-value function satisfies:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$$

The Bellman optimality equation can be expressed without reference to a specific policy π , instead selecting the best possible action. The Bellman optimality equation for the optimal state-value function is:

$$v^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r \mid s, a) [r + \gamma v^*(s')]$$

It states that the optimal value of a state is the maximum expected total reward achievable by choosing the best action, and assuming optimal behaviour in subsequent states. The Bellman optimality equation for the optimal action-value function states the optimal value of a action-state pair:

$$q^*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right]$$

It represents the maximum expected total reward obtained by taking action a in state s , followed by optimal behaviour thereafter. It forms the foundation for Q-learning algorithm.

2.4. Q-Learning

A model of the environment refers to any mechanism that allows the agent to predict the outcome of its actions. Reinforcement learning methods can be categorized into model-based and model-free methods, depending on whether the agent has access to a model of the environment. Model-based agents utilize planning, which is a computational process that uses a model to improve a policy. The agent uses its model to generate simulated experience and evaluate future trajectories. In contrast, model-free agents learn directly from real experience by interacting with the environment, executing actions and observing outcomes.

Q-learning is categorized as a model-free, off-policy algorithm. The primary goal of off-policy learning is to evaluate and optimize a target policy, which is

the strategy intended to become optimal. Simultaneously, the agent follows a distinct, more exploratory behaviour policy to interact with the environment. The ϵ -greedy policy is used in Q-learning to select an action, which balances the trade-off between exploration and exploitation. With probability of ϵ , the agent explores by selecting a random action (or the non-greedy action) to improve its estimates of the optimal values. With probability $1 - \epsilon$, the agent exploits its current knowledge by choosing the greedy action, which is the action with the highest current estimated value.

Q-learning is a Temporal-Difference (TD) control algorithm that directly approximates the optimal action-value function. TD error, δ_t , represents the discrepancy between the current estimate of Q -value and the updated estimate of future value from the next state:

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, a') - Q(S_t, A_t)$$

The Q-learning update rule utilizes this error to refine its estimates:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

where $\alpha \in [0, 1]$ is the learning rate (or step-size parameter). The Q -value $Q(S_t, A_t)$ is the current estimate of the value of taking action A in state S . It is updated by incorporating the immediate reward and the discounted value of the best possible action in the subsequent state. This update assumes that the agent follows the optimal policy from the next state onward. As the estimates converge to the optimal values (Q^*), the optimal policy π^* can be obtained by selecting the action that maximizes this value: $\pi^*(x) = \arg \max_a Q^*(x, a)$.

3. Methods

- **Christina and Caleb ## Implementation** We built and implemented a Q-learning algorithm within a maze environment. The environment was represented as a 6×6 grid constructed using a NumPy array, where integer values encoded state types and associated rewards. Specifically, cells were defined as follows: 0 represented empty traversable space, 1 represented impassable walls, 2 represented fatal states (“holes”) that returned the agent to the starting position, and 3 represented the goal state associated with a positive reward and episode termination.

A Q-table was initialized with zeros to represent the expected value of each state-action pair. The learning rate (), future discount factor (), and exploration rate () were predefined and held constant throughout training. The agent’s action space consisted of four discrete movements (up, down, left, right).

At each step, the agent selected an action using the Epsilon-Greedy strategy. State transitions were determined by applying the selected action to the agent’s current position, with resulting coordinates mapped onto the grid to determine the subsequent state and corresponding reward.

Q-values were iteratively updated using the temporal difference learning rule based on the observed reward and the maximum expected future reward from the subsequent state. Training was conducted over a fixed number of episodes, with each episode beginning at the starting state and terminating upon reaching the goal or a terminal penalty state.

3.1. Neuropsychiatry Extension

- Caleb Part
- Q learning in computational psychiatry
- Example: D1/D2 receptors

4. Results

4.1. Model

After running 1000 episodes, we present evidence of the agent’s learning process. Figure 1 illustrates changes in the number of steps the agent takes to complete one episode over the course of 500 episodes.

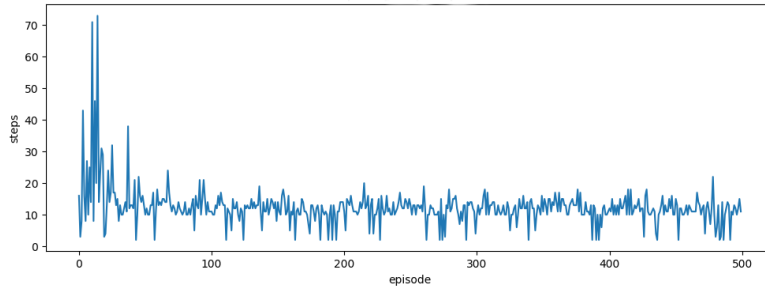


Figure 1: Changes in Steps to Complete One Episode

Figure 2 presents changes in the TD learning error (delta) over time across 6000 episodes.

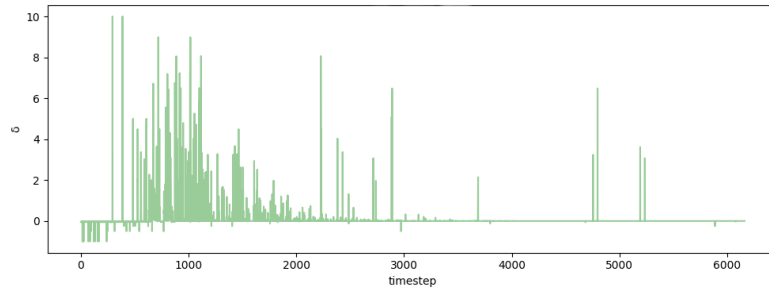


Figure 2: TD Error Over Time

Figure 3 is a Q-value heatmap showing the Q values of the best action to take in a given state (i.e. state-action pair that renders the maximal Q-value), with greater Q-values of darker color.

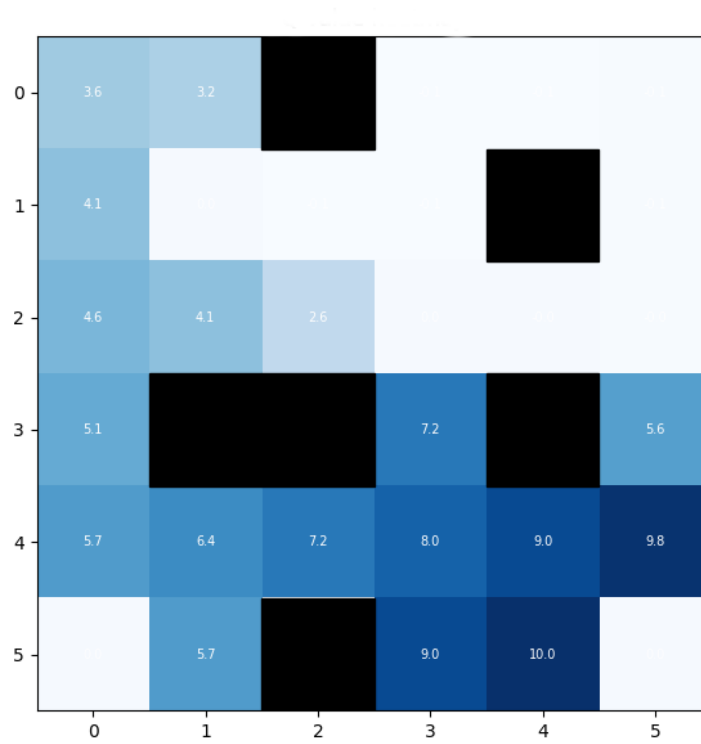


Figure 3: Q-Value Heatmap

Figure 4 illustrates the optimal actions to take in a given state, with visualizations of actions as arrows in the maze environment. This represents the optimal policy the agent is learning.

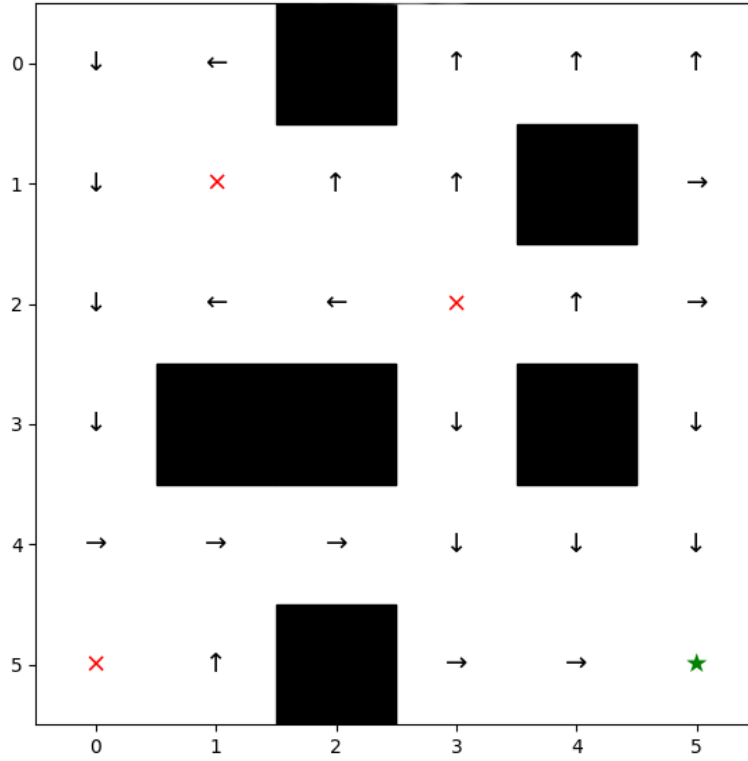


Figure 4: Optimal Learned Policy

4.2. Neuropsychiatry Extension

- Caleb

References

- Dirac, P.A.M., 1953. The Lorentz transformation and absolute time. *Physica* 19, 888–896. doi:[10.1016/S0031-8914\(53\)80099-6](https://doi.org/10.1016/S0031-8914(53)80099-6).
- Feynman, R.P., Vernon Jr., F.L., 1963. The theory of a general quantum system interacting with a linear dissipative system. *Annals of Physics* 24, 118–173. doi:[10.1016/0003-4916\(63\)90068-X](https://doi.org/10.1016/0003-4916(63)90068-X).